Responses to the editorial comments on the previous submission "RprobitB: Bayes Estimation of Choice Behavior Heterogeneity in R" (JSS 4817), which accompany the new submission "RprobitB: Bayesian Probit Choice Models with Preference Heterogeneity in R"

We thank the editors for their comments on our previous submission. We have since rewritten the manuscript and revised the R package {RprobitB} (Oelschläger and Bauer 2026). Below, we quote each comment and state how it is addressed. The software changes are listed at the end of this document.

Responses to the editorial report:

1. Comment: The introduction indicates that a number of other software packages provide related functionality. It would seem that the specific added functionality of RprobitB compared to other packages is "LC" and "LC update", i.e., weight-based class updates as well as a Dirichlet process mixture approach. However, this aspect of the implementation is not sufficiently well discussed or presented using applications to be convincing.

   Response: We now state the contribution of the package more clearly and have improved the comparison with competing packages. We also discuss the latent class models in more detail and have reworked their application.

   Section 1 describes how competing packages model preference heterogeneity. The selection is guided by two central aspects of {RprobitB}, namely probit models and preference heterogeneity. For preference heterogeneity, we include the established open-source packages {mlogit}, {gmnl}, {logitr}, {apollo}, and Biogeme, which model it mostly via logit models. For probit models, we include {bayesm} and {MNP} for Bayesian estimation, {Rchoice} for random coefficients in binary and ordered models, and {Rprobit}, the frequentist counterpart of {RprobitB}. Table 1 shows for each of them whether it offers probit models, Bayesian estimation, random coefficients, latent classes, and inference on the number of classes.

   As Table 1 shows, {RprobitB} is, to the best of our knowledge, the only package that estimates multinomial probit models with random coefficients and correlated errors in a Bayesian framework, and the only one that infers the number of latent classes of such a model from the data; {bayesm} does so for logit models. The added functionality is not limited to the latent classes: for these probit models, the package offers normal and log-normal mixing distributions (Sections 2.4, 5.1, and 5.3), inference on individual deciders and conditional predictions (Sections 5.2 and 7.1), and model comparison (Section 8).

The latent class functionality is now discussed in detail in Sections 2.5, 3.2, and 3.3 (see our responses to the next three comments). The application in Section 5.4 now uses the `Electricity` data, for which a single normal mixing distribution cannot represent a skewed or multimodal valuation of a local supplier. The Dirichlet process mixture and the sparse finite mixture infer the number of occupied classes, and `latent_class_diagnostics()` reports its posterior distribution. The weight-based heuristic is demonstrated on simulated data with a trimodal mixing distribution (Figure 5). Further applications cover a stated choice experiment on train trips (Sections 3.5, 4, 5.3, 7.3, and 8), the `Health` data for ordered responses (Sections 6.1 and 7.2), the `Game` data for ranked responses (Section 6.2), and Berserk decisions in an online chess tournament for conditional predictions (Section 7.1).

2. Comment: The weighted-based class updates approach seem to have only been proposed by the authors in a conference publication and still lacks a sufficient statistical basis. It would seem that this is a rather ad-hoc approach for estimating the number of classes requiring various tuning parameters where no insights are provided for the reader how to choose or select them and what their impact is from a theoretical perspective regarding a constraint estimation or from a Bayesian point of view how this could be translated to a prior.

   Response: We agree that the weight-based heuristic lacks a formal statistical basis. It is therefore no longer the main contribution of the package, and the manuscript presents it as one of four treatments of the number of classes, which the argument `class_update` of `fit()` selects (Sections 2.5 and 3.3): (i) a fixed number of classes (`"fixed"`); (ii) an overfitted finite mixture with a sparse Dirichlet prior on the class weights (Rousseau and Mengersen 2011; Malsiner-Walli, Frühwirth-Schnatter, and Grün 2016; Frühwirth-Schnatter and Malsiner-Walli 2019) (`"sparse"`); (iii) a Dirichlet process mixture (Escobar and West 1995), sampled by Algorithm 8 of Neal (2000) (`"dirichlet_process"`); and (iv) the heuristic of Oelschläger and Bauer (2021) (`"weight_based"`).

   From a Bayesian point of view, options (ii) and (iii) address the same task through a prior. In the sparse finite mixture, a sparse prior on the class weights empties the classes that the data do not require, similar to the removal of small classes by the heuristic, and the Dirichlet process mixture makes the number of occupied classes part of the posterior. We therefore recommend these options for inference on latent classes, and the heuristic primarily for the approximation of non-normal mixing distributions by a mixture of normal distributions (Oelschläger 2026b). We do not attach a prior interpretation to the heuristic. It changes the number of classes only during the warmup (Section 5.4). Afterwards, each chain keeps the

number of classes it has reached. The retained draws thus come from an ordinary finite mixture with this number of classes and do not reflect the uncertainty about the number of classes, which options (ii) and (iii) capture.

Section 5.4 states the main tuning parameters and their defaults, which worked well in our simulations: a class is removed if its weight falls below `epsmin = 0.01` and split if its weight exceeds `epsmax = 0.7`, two classes are merged if their means are closer than `deltamin = 0.1`, and at most one change occurs per `buffer = 50` iterations. Suitable values also depend on the specific application, which makes general recommendations difficult. Larger values of `epsmin` and `deltamin` tend to reduce the number of classes. Since only splits add classes, no class is added while all weights stay below `epsmax`, and a smaller `epsmax` tends to increase the number of classes. In a simulation with three equally weighted classes, the heuristic converges to three classes when starting from one and from ten initial classes (Figure 5).

3. Comment: Related work on finite and infinite mixture models would indicate that in particular the Dirichlet parameter delta for finite mixtures and the precision parameter for DPMs would seem influential (see for example Frühwirth-Schnatter and Malsiner-Walli, 2019). In the current exposition it remains unclear how the model could be estimated if the number of classes should be a-priori specified to be filled and for example model-selection based on information criteria performed in the finite case, or a sparse finite mixture approach pursued or how the precision parameter is to be selected in DPMs. It would seem important to provide more theoretical as well as practical guidance on how to specify and apply the mixture approach in practice.

   Response: The manuscript now addresses the three approaches mentioned in the comment, both methodologically (Sections 2.5, 3.2, and 3.3) and in an application to the same data (Sections 5.4 and 8.2). As Section 3.2 and Table 4 state, the class weights of a finite mixture have a symmetric Dirichlet prior with concentration parameter δ, which by default equals 1 for `"fixed"` and `"weight_based"`, making the prior uniform on the simplex.

   With `class_update = "fixed"`, the number of classes is specified a priori, and the sampler targets the posterior restricted to allocations without empty classes (Section 3.3). Model selection then compares fits with different numbers of classes: `AIC()` and `BIC()` use `logLik()`; `WAIC()` and `loo()` compute WAIC and PSIS-LOO at the decider level; and `bayes_factor()` estimates Bayes factors (Section 8). For the `Electricity` data, PSIS-LOO favors two classes over one.

   With `class_update = "sparse"`, the specified number of classes is an upper bound, and δ has a gamma hyperprior with shape 1 and rate 200, as in Frühwirth-Schnatter

and Malsiner-Walli (2019). With six potential classes, the sparse finite mixture places a posterior probability of 91 percent on two occupied classes (Section 5.4).

With `class_update = "dirichlet_process"`, δ is the concentration, or precision, parameter of the process. The sampler updates it by the augmentation of Escobar and West (1995) under their gamma hyperprior with shape 2 and rate 4. For the 361 households, the prior expected number of occupied classes is about 3.9 at the prior mean $\delta = 0.5$ (Section 5.4), and the posterior places 47 percent on two classes and 53 percent on three or more. For the sparse finite mixture with six potential classes, the prior expected number of occupied classes is only about 1.2 at the prior mean $\delta = 0.005$. Under the matching of Frühwirth-Schnatter and Malsiner-Walli (2019), this corresponds to a Dirichlet process whose concentration has prior mean 0.03 instead of 0.5. With this matched hyperprior, the Dirichlet process places 87 percent on two classes, close to the 91 percent of the sparse finite mixture. The following code reproduces it from `electricity_classes` of Section 5.4:

```
set.seed(1)
matched <- update(electricity_classes,
  class_update = "dirichlet_process",
  prior = list(class_concentration = c(shape = 1, rate = 200 / 6)))
subset(latent_class_diagnostics(matched)$occupancy, n_classes == 2)
#>   n_classes probability
#> 2         2   0.8686667
```

The difference between the two mixtures is thus mainly due to their hyperpriors, in line with the finding of Frühwirth-Schnatter and Malsiner-Walli (2019) that the hyperprior influences the cluster solution far more than the choice between the two mixtures. In practice, we recommend checking the number of occupied classes under a second hyperprior, which can be set through `prior`.

4. Comment: Label switching also seems to be only resolved using a constraint on the class weights. Given the large literature on resolving this issue for Bayesian mixture models and the failure of using only a constraint on one specific parameter in some cases this seems not satisfactory.

   Response: The constraint on the class weights has been removed. Following the literature on label switching (e.g., Stephens 2000), the sampler imposes no ordering constraint, and the retained draws are relabeled after sampling (Section 3.3). A reference allocation is selected by the least-squares criterion of Dahl (2006) with respect to the posterior co-clustering matrix, and each draw is permuted to agree with it by the equivalence classes representatives algorithm of Papastamoulis and

Iliopoulos (2010). The reference is then reset to the most frequent relabeled class of every decider until it is stable, and all class-specific parameters and the allocations are permuted accordingly. Since the relabeling is based on the allocations of all deciders, it does not require the classes to be separated in one particular parameter, whereas an ordering constraint does. The classes are then numbered by decreasing posterior mean weight. In the simulation of Section 3.4, whose three classes have equal weights and therefore cannot be identified by an ordering constraint on the weights, the relabeled class weights and means are close to their true values, with a rank-normalized $\hat{R}$ (Vehtari et al. 2021) of at most 1.01.

5. Comment: We decided not to send the submission into full review because in its current form the chances for successful publication are deemed too low. The package provides a nice set of functionality making use of R's classes and methods system and building on infrastructure available, e.g., for formulas. However, it is not sufficiently well outlined how the package provides additional functionality compared to other packages and the potential additional functionality is not well enough theoretically explained, the statistical details and their implementation presented and their use illustrated in practice.

   Response: The package retains its S3 classes and methods and its formula interface. The additional functionality compared to other packages is outlined in Section 1 and Table 1 (see our response to the first comment). It is explained theoretically in Section 2, which defines the model class with random coefficients, latent classes, and ordered and ranked responses, together with the likelihood. The statistical details and their implementation are presented in Section 3: the function `fit()` and its arguments (Section 3.1 and Table 3), the prior distributions with their defaults (Section 3.2 and Table 4), the main blocks of the Gibbs sampler, the treatment of the number of classes, and the relabeling (Section 3.3). A simulation checks whether the sampler recovers the parameters of a known data generating process (Section 3.4). The use in practice is demonstrated in the applications listed in our response to the first comment, always with code illustrations, from the interpretation of the estimates (Section 4) to model evaluation (Section 8).

Responses to the specific comments:

1. Comment: The NAMESPACE contains

   ```
   S3method(AIC,RprobitB_fit) S3method(BIC,RprobitB_fit)
   ```

   raising the question why the logLik method does not return a logLik object where AIC and BIC methods are readily available for.

Response: `logLik()` now returns a `logLik` object with the number of parameters and the number of observations as attributes. The functions `AIC()` and `BIC()` of {`stats`} therefore apply directly, and the package no longer defines its own `AIC` and `BIC` methods. Section 8.1 applies `logLik()`, which evaluates the log-likelihood at the posterior means of the population parameters.

2. Comment: The harmonic mean estimator has been pointed out to be problematic for estimating the marginal likelihood. This aspect should be also discussed and covered.

   Response: We agree and have removed the harmonic mean estimator from the package. As Frühwirth-Schnatter (2004) points out, it is a reciprocal importance sampling estimator with the prior as importance density and therefore unreliable whenever the prior has heavier tails than the posterior. The marginal likelihood is now estimated by bridge sampling (Meng and Wong 1996), which is much more robust to the tail behavior of the importance density (Frühwirth-Schnatter 2004). Section 8.3 applies it to the train models, with a result in line with the PSIS-LOO comparison of Section 8.2. For models without latent classes, the package uses {`bridgesampling`} (Gronau, Singmann, and Wagenmakers 2020) with a normal importance density fitted to the population parameters on an unconstrained scale. Since a single normal importance density is unsuitable for the multimodal posterior of a mixture (Frühwirth-Schnatter 2004), the importance density of a model with a fixed number of up to six latent classes averages a normal density, fitted to the relabeled draws, over all permutations of the class labels, following the idea of full permutation bridge sampling (Celeux, Frühwirth-Schnatter, and Robert 2019; Frühwirth-Schnatter 2019).

Changes from {`RprobitB`} 1.1.0 to {`RprobitB`} 2.0.1:

- The single function `fit()` replaces the multi-step workflow. It accepts data in wide or long layout and simulates data if `data = NULL`. Data handling and simulation are provided by the companion package {`choicedata`} (Oelschläger 2026a).

- Random effects are specified by name through `random_effects`, with correlated or uncorrelated normal, log-normal, and negative log-normal mixing distributions. The argument `latent_class_effects` names the effects that differ between the latent classes.

- With `class_update`, the number of classes is fixed or adapted during the warmup by the weight-based heuristic, or a sparse finite or a Dirichlet process mixture infers the number of occupied classes. The function `latent_class_diagnostics()` summarizes its posterior and the class memberships.

- `interpret()` computes compensations between effects, such as the willingness to pay, and marginal effects, each with posterior uncertainty.
- `predict()` accounts for posterior uncertainty and provides conditional predictions given the choice history of a decider. `residuals()` is new, and `update()` is adapted to `fit()`.
- The retained draws are stored as a `draws_array` object of {`posterior`}. The function `summary()` reports the convergence diagnostics of Vehtari et al. (2021), `plot()` uses {`bayesplot`}, and chains can run in parallel via {`future`}.
- The C++ implementation of the Gibbs sampler is more efficient. For instance, the models of Sections 3.5 and 5.1 run about two times faster than with version 1.1.0.

References:

Celeux, Gilles, Sylvia Frühwirth-Schnatter, and Christian P. Robert (2019). "Model Selection for Mixture Models – Perspectives and Strategies". In: *Handbook of Mixture Analysis*. Ed. by Sylvia Frühwirth-Schnatter, Gilles Celeux, and Christian P. Robert. Chapman and Hall/CRC, pp. 117–154. DOI: 10.1201/9780429055911-7.

Dahl, David B. (2006). "Model-Based Clustering for Expression Data via a Dirichlet Process Mixture Model". In: *Bayesian Inference for Gene Expression and Proteomics*. Ed. by Kim-Anh Do, Peter Müller, and Marina Vannucci. Cambridge University Press, pp. 201–218. DOI: 10.1017/CBO9780511584589.011.

Escobar, Michael D. and Mike West (1995). "Bayesian Density Estimation and Inference Using Mixtures". In: *Journal of the American Statistical Association* 90.430, pp. 577–588. DOI: 10.1080/01621459.1995.10476550.

Frühwirth-Schnatter, Sylvia (2004). "Estimating Marginal Likelihoods for Mixture and Markov Switching Models Using Bridge Sampling Techniques". In: *The Econometrics Journal* 7.1, pp. 143–167. DOI: 10.1111/j.1368-423X.2004.00125.x.

— (2019). "Keeping the Balance – Bridge Sampling for Marginal Likelihood Estimation in Finite Mixture, Mixture of Experts and Markov Mixture Models". In: *Brazilian Journal of Probability and Statistics* 33.4, pp. 706–733. DOI: 10.1214/19-BJPS446.

Frühwirth-Schnatter, Sylvia and Gertraud Malsiner-Walli (2019). "From Here to Infinity: Sparse Finite versus Dirichlet Process Mixtures in Model-Based Clustering". In: *Advances in Data Analysis and Classification* 13.1, pp. 33–64. DOI: 10.1007/s11634-018-0329-y.

Gronau, Quentin F., Henrik Singmann, and Eric-Jan Wagenmakers (2020). "bridgesampling: An R Package for Estimating Normalizing Constants". In: *Journal of Statistical Software* 92.10, pp. 1–29. DOI: 10.18637/jss.v092.i10.

Malsiner-Walli, Gertraud, Sylvia Frühwirth-Schnatter, and Bettina Grün (2016). "Model-Based Clustering Based on Sparse Finite Gaussian Mixtures". In: *Statistics and Computing* 26.1–2, pp. 303–324. DOI: 10.1007/s11222-014-9500-2.

Meng, Xiao-Li and Wing Hung Wong (1996). "Simulating Ratios of Normalizing Constants via a Simple Identity: A Theoretical Exploration". In: *Statistica Sinica* 6.4, pp. 831–860. URL: https://www3.stat.sinica.edu.tw/statistica/j6n4/j6n43/j6n43.htm.

Neal, Radford M. (2000). "Markov Chain Sampling Methods for Dirichlet Process Mixture Models". In: *Journal of Computational and Graphical Statistics* 9.2, pp. 249–265. DOI: 10.1080/10618600.2000.10474879.

Oelschläger, Lennart (2026a). *choicedata: Working with Choice Data*. R package version 0.2.1. DOI: 10.32614/CRAN.package.choicedata.

— (2026b). "Overcoming Challenges in Modeling Choice Behavior Heterogeneity". PhD thesis. Bielefeld, Germany: Bielefeld University. URL: https://pub.uni-bielefeld.de/record/3014719.

Oelschläger, Lennart and Dietmar Bauer (2021). "Bayes Estimation of Latent Class Mixed Multinomial Probit Models". In: *Proceedings of the 100th Annual Meeting of the Transportation Research Board*. Washington, DC. URL: https://trid.trb.org/view/1759753.

— (2026). *RprobitB: Bayesian Probit Choice Modeling*. R package version 2.0.1. DOI: 10.32614/CRAN.package.RprobitB.

Papastamoulis, Panagiotis and George Iliopoulos (2010). "An Artificial Allocations Based Solution to the Label Switching Problem in Bayesian Analysis of Mixtures of Distributions". In: *Journal of Computational and Graphical Statistics* 19.2, pp. 313–331. DOI: 10.1198/jcgs.2010.09008.

Rousseau, Judith and Kerrie Mengersen (2011). "Asymptotic Behaviour of the Posterior Distribution in Overfitted Mixture Models". In: *Journal of the Royal Statistical Society B* 73.5, pp. 689–710. DOI: 10.1111/j.1467-9868.2011.00781.x.

Stephens, Matthew (2000). "Dealing with Label Switching in Mixture Models". In: *Journal of the Royal Statistical Society B* 62.4, pp. 795–809. DOI: 10.1111/1467-9868.00265.

Vehtari, Aki et al. (2021). "Rank-Normalization, Folding, and Localization: An Improved $\widehat{R}$ for Assessing Convergence of MCMC (with Discussion)". In: *Bayesian Analysis* 16.2, pp. 667–718. DOI: 10.1214/20-BA1221.